# AprilTag ID Sorting

## 1. Function Description

After the program runs, the robotic arm will move to its initial position, then place the 3cm high AprilTag wooden block from the tutorial within the camera's view. Press the spacebar, and the robotic arm will lower its gripper to grasp the recognized AprilTag and place it at the designated position based on the AprilTag's ID value.

## 2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start AprilTag recognition node
ros2 run dofbot_sorting apriltag_sorting
# Start grasping node
ros2 run dofbot_sorting grasp
```

After successfully running the program, the robotic arm will move to its initial position, then place the 3cm high AprilTag within the camera's view as shown in the figure below. Press the spacebar, and the robotic arm will lower its gripper to grasp the recognized AprilTag. Then, based on the ID of the grasped AprilTag, it will place it at the designated position.

# 3. Code Analysis

## 3.1. apriltag_sorting

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/apriltag_sorting.py
```

Import necessary library files:

```
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
from dofbot_sorting.vutils import draw_tags
from dofbot_interface.msg import *
from std_msgs.msg import Float32,Bool,Int16
import time
import math
from Arm_Lib import Arm_Device
import os
#compute_joint5 is a library for calculating joint 5 angle. Source code location
is at /root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/compute_joint5.py
from dofbot_sorting.compute_joint5 import *
```

Create robotic arm control object and control the robotic arm to move to initial position:

```
self.Arm = Arm_Device()
self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Create publishers and subscribers:

```
# Create publisher to publish recognized position information topic messages
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)
# Create publisher to publish calculated joint 5 angle reference value topic
messages
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)
# Create subscriber to subscribe to grasp completion topic messages
self.subscription =
self.create_subscription(Bool,'grasp_done',self.GraspStatusCallback,qos_profile=
1)
# Create subscriber to subscribe to image topic messages
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Grasp completion topic callback function GraspStatusCallback:

```
# Process grasp completion topic messages. When the received message is True, it indicates the current sorting is completed and the next sorting can be performed. Change self.pubPos_flag to indicate that AprilTag information can be published for the next time.
def GraspStatusCallback(self,msg):
    print("**")
    if msg.data == True:
        self.pubPos_flag = True
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages, convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    result_image = np.copy(rgb_image)
    # Perform AprilTag recognition
    tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY),
False, None, 0.025)
    tags = sorted(tags, key=lambda tag: tag.tag_id)
    # Draw AprilTag information on the image
    draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0,
255, 0))
    # Detect key press. If spacebar is pressed (spacebar value is 32), modify
self.pubPos_flag value to True, indicating that AprilTag information topic
messages can be published
    key = cv2.waitKey(10)
    if key == 32:
        self.pubPos_flag = True
    # If tags length is greater than 0, it means AprilTag has been recognized
    if len(tags) > 0 :
        # Traverse the AprilTag list
        for i in range(len(tags)):
            # Extract AprilTag center point coordinates
            center_x, center_y = tags[i].center
            cx = center_x
            cy = center_y
            # Convert pixel coordinates to normalized coefficients a and b. a is
the x-axis offset coefficient, b is the y-axis offset coefficient. The calculated
ba values are the center point coordinates in the world coordinate system, which
will eventually be sent to the sorting grasp node.
            (a, b) = (round(((320 - cx) / 4000), 5), round(((480 - cy) / 3000) *
0.8+0.15, 5))
            print("a: ",a)
            print("b: ",b)
            # Extract AprilTag corner point coordinates
            vx = int(tags[i].corners[0][0]) - int(tags[i].corners[1][0])
            vy = int(tags[i].corners[0][1]) - int(tags[i].corners[1][1])
            # Calculate joint 5 angle reference value
            target_joint5 = compute_joint5(vx,vy)
            print("target_joint5: ",target_joint5)
            if self.pubPos_flag == True:
                # Set self.pubPos_flag to False to prohibit sending position
topics, preventing sending recognized position messages again during grasping
process
                self.pubPos_flag = False
                # Create joint 5 angle reference value topic message object
```

```
                joint5 = Int16()
                # Assign data to joint 5 angle reference value topic object
                joint5.data = int(target_joint5)
                self.pub_joint5.publish(joint5)
                # Create position information topic message object
                pos = AprilTagInfo()
                # Assign data to position information topic message object
                pos.x = b
                pos.y = a
                # z is the height value of AprilTag in meters. If AprilTag height
is changed, this needs to be modified accordingly
                pos.z = 0.03
                # pos.id represents the AprilTag's id value
                pos.id = tags[i].tag_id
                # Publish position information topic
                self.pos_info_pub.publish(pos)
```

### 3.2. grasp

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/grasp.py
```

Import necessary library files:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32, Bool,Int16
import time
from dofbot_interface.msg import *       # Need to confirm if ROS2 message
package name is consistent
from dofbot_interface.srv import Kinemarics
import threading
from Arm_Lib import Arm_Device
```

Create robotic arm control object:

```
self.Arm = Arm_Device()
```

Create subscribers and publishers:

```
# Create subscriber to subscribe to position information topic
self.sub = self.create_subscription(AprilTagInfo,'PosInfo',self.pos_callback,1)
# Create subscriber to subscribe to joint 5 angle reference value topic
self.sub_joint5 =
self.create_subscription(Int16,"set_joint5",self.get_joint5_callback,1)
# Create subscriber to subscribe to joint 6 (gripper) angle value topic
self.sub_joint6 =
self.create_subscription(Int16,"set_joint6",self.get_joint6_callback,1)
# Create publisher to publish sorting grasp completion topic
self.pubGraspStatus = self.create_publisher(Bool, 'grasp_done',1)
```

Create inverse kinematics service client:

```
self.client = self.create_client(Kinemarics, 'dofbot_kinemarics')
```

Joint 5 angle reference value topic callback function:

```
def get_joint5_callback(self,msg):
    self.joint_5 = msg.data
```

Joint 6 (gripper) angle value topic callback function:

```
def get_joint6_callback(self,msg):
    self.joint_6 = msg.data
```

Position information topic callback function:

```
def pos_callback(self,msg):
    pos_x = msg.x
    pos_y = msg.y
    pos_z = msg.z
    # Record current ID, later select placement position based on self.cur_tagId
value
    self.cur_tagId = msg.id
    print("msg: ",msg)
    # If self.grasp_flag value is True, grasping can be performed
    if self.grasp_flag == True :
        print("Take it now.")
        # Change self.grasp_flag value to False to prevent repeatedly entering
this judgment during grasping process
        self.grasp_flag = False
        # Create a thread to execute grasp function, passing parameters
pos_x,pos_y,pos_z
        threading.Thread(target=self.grasp, args=(pos_x,pos_y,pos_z)).start()
```

grasp function:

```
def grasp(self,pos_x,pos_y,pos_z):
    print("--------------------------------------------------")
    # Call inverse kinematics service
    request = Kinemarics.Request()
    # 0.01 is forward/backward grasping point offset. The offset can be set
according to grasping effect. If grasping is too forward, reduce the offset; if
too backward, increase the offset. Offset value unit is meters
    request.tar_x = pos_x -0.01
    # 0.02 is left/right grasping point offset. The offset can be set according
to grasping effect. If grasping is too left, reduce the offset; if too right,
increase the offset. Offset value unit is meters
    request.tar_y = pos_y - 0.02
    # 0.03 is up/down grasping point offset. The offset can be set according to
grasping effect. If grasping is too low, reduce the offset; if too high, increase
the offset. Offset value unit is meters
    request.tar_z = pos_z + 0.03
    request.kin_name = "ik"
    # We expect the grasping posture to rotate 60 degrees around the y-axis, so
we give 1.04 radians
    request.pitch = 1.04
```

```
    try:
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
        response = future.result()
        print("calcutelate_response: ",response)
        joints = [0.0, 0.0, 0.0, 0.0, 0.0,0.0]
        # Get angle values from inverse kinematics feedback
        joints[0] = response.joint1 #response.joint1
        joints[1] = 180 - response.joint2
        joints[2] = 180 - response.joint3
        joints[3] = 180 - response.joint4
        print("self.joint_5: ",self.joint_5)
        target_joint5 = self.joint_5
        # Calculate actual angle value based on obtained joint 5 angle reference
value
        if target_joint5>0 and target_joint5 < 45:
            joint5 = 90 - target_joint5
        elif target_joint5>45 and target_joint5<135:
            joint5 = target_joint5
        elif target_joint5>=135:
            joint5 = target_joint5 - 90
        elif target_joint5>-45 and target_joint5<0:
            joint5 = 90 + abs(target_joint5)
        elif target_joint5<-45 and target_joint5>-135:
            joint5 = abs(target_joint5)
        elif target_joint5<-135:
            joint5 = abs(target_joint5) - 90
        self.set_joint5 = 180 - joint5
        joints[5] = 30
        # Call robotic arm control object function to control six servos to move
to specified posture

 self.Arm.Arm_serial_servo_write6(joints[0],joints[1],joints[2],joints[3],90,joi
nts[5],2000)
        time.sleep(3.5)
        # Execute move function
        self.move()
    except Exception:
       pass
```

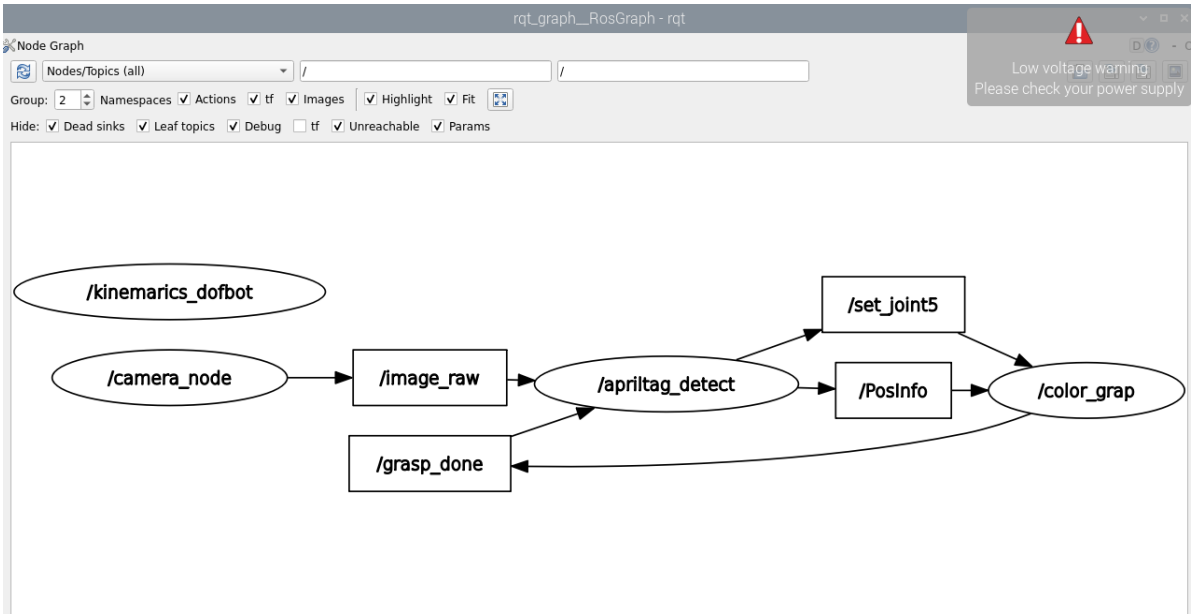
move function:

```
def move(self):
    # Control joint 5 to move to calculated angle
    self.Arm.Arm_serial_servo_write(5, self.set_joint5, 2000)
    time.sleep(2.0)
    # Control joint 6 to grip. If grip is not tight enough, increase this
parameter. If grip is too tight causing joint 6 stall, then reduce this value
    self.Arm.Arm_serial_servo_write(6, 140, 2000)
    time.sleep(2.0)
    # Control joint 2 to lift up
    self.Arm.Arm_serial_servo_write(2, 120, 2000)
    time.sleep(2.0)
    print("self.cur_tagId",self.cur_tagId)
    # Select self.down_joint (placement position) based on self.cur_tagId
    if self.cur_tagId == 1:
        self.down_joint = [155.0, 45, 70, 5, 60.0,self.joint_6]
```

```
    elif self.cur_tagId == 2:
        self.down_joint = [180.0, 45, 60, 0, 90.0,self.joint_6]
    elif self.cur_tagId == 3:
        self.down_joint = [28.0, 40, 70, 2, 90.0,self.joint_6]
    elif self.cur_tagId == 4:
        self.down_joint = [0.0, 53, 48, 6, 90.0,self.joint_6]
    elif self.cur_tagId == 0:
        self.down_joint = [155.0, 45, 70, 5, 60.0,self.joint_6]
    print("self.down_joint",self.down_joint)
    # Control six servos to move to placement posture
    self.Arm.Arm_serial_servo_write6_array(self.down_joint,2000)
    time.sleep(2.0)
    # Control joint 6 to release
    self.Arm.Arm_serial_servo_write(6, 90, 2000)
    time.sleep(2.0)
    self.Arm.Arm_serial_servo_write(2, 90, 2000)
    time.sleep(2.0)
    # Control six servos to move to initial posture for next recognition and
grasping
    self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
    time.sleep(4)
    # Modify self.grasp_flag to True, indicating that the next grasping action
can be performed
    self.grasp_flag = True
    # Create grasp completion topic message object
    grasp_done = Bool()
    # Assign data to grasp completion topic message object
    grasp_done.data = True
    # Publish grasp completion topic
    self.pubGraspStatus.publish(grasp_done)
```

## 4. Node Communication Diagram

After entering the docker container, enter the following commands in the docker terminal:

```
ros2 run rqt_graph rqt_graph
```

Then select all nodes in the upper left corner as shown in the figure below: